

CS2569/.....

เค้าโครงโครงงานคอมพิวเตอร์

ระบบตรวจสอบสภาวะเครือข่ายแบบเรียลไทม์และการตรวจจับสิ่งผิดปกติของโปรโตคอล TCP ในระดับเคอร์เนลด้วย eBPF และภาษา Go

Real-time Network Observability and TCP Anomaly Detection System at Kernel-Level Using eBPF and Go

โดย

673380275-5 นางสาวปภาวรินทร์ นาเมืองรักษ์

673380300-2 นายเมธัส มณีวิจิตร

อาจารย์ที่ปรึกษา ผศ.บุญทรัพย์ ไวคำ

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353761 สัมมนาทางวิทยาการคอมพิวเตอร์

ภาคเรียนที่ 1 ปีการศึกษา 2569

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

(เดือน กันยายน พ.ศ. 2569)

การเสนอเค้าโครงโครงงานคอมพิวเตอร์

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

ชื่อ นางสาวปภาวรินทร์ นาเมืองรักษ์ รหัสประจำตัว 673380275-5

Miss Phapawarin Namuangrak

ชื่อ นายเมธัส มณีวิจิตร รหัสประจำตัว 673380300-2

Mr. Methas Maneewijit

นักศึกษาระดับปริญญาตรี หลักสูตรวิทยาการคอมพิวเตอร์

อาจารย์ที่ปรึกษาโครงงาน ผศ.บุญทรัพย์ ไวคำ

Project Advisor: Asst. Prof. Boonsup Waikham

1. ชื่อหัวข้อโครงงาน

ภาษาไทย ระบบตรวจสอบสภาวะเครือข่ายแบบเรียลไทม์และการตรวจจับสิ่งผิดปกติของโปรโตคอล TCP ในระดับเคอร์เนลด้วย eBPF และภาษา Go

ภาษาอังกฤษ Real-time Network Observability and TCP Anomaly Detection System at Kernel-Level Using eBPF and Go

2. หลักการและเหตุผล

โปรโตคอล TCP (Transmission Control Protocol) เป็นกลไกหลักในการสื่อสารบนเครือข่ายคอมพิวเตอร์ การติดตามเหตุการณ์ของการเชื่อมต่อ TCP เช่น การสร้างการเชื่อมต่อ (Handshake) การเปลี่ยนสถานะ การส่งข้อมูลซ้ำ (Retransmission) และการปิดการเชื่อมต่อ มีความจำเป็นต่อการตรวจจับพฤติกรรมที่ไม่พึงประสงค์บนเครือข่าย อย่างไรก็ตาม แนวทางดั้งเดิมที่ดักจับและวิเคราะห์ข้อมูลระดับแพ็กเก็ต (Packet-level Inspection) มักก่อให้เกิดภาระการประมวลผลสูงและมีปริมาณข้อมูลมาก ซึ่งเป็นอุปสรรคต่อการทำงานแบบเรียลไทม์บนเครือข่ายที่มีปริมาณการจราจร(traffic)สูง (Liu et al., 2020; Scholz et al., 2018)

eBPF (extended Berkeley Packet Filter) เป็นเทคโนโลยีที่อนุญาตให้โปรแกรมขนาดเล็กทำงานภายในเคอร์เนลของ Linux ได้อย่างปลอดภัยผ่านกลไกตรวจสอบของ Verifier โดยไม่ต้องแก้ไขซอร์สโค้ดของเคอร์เนล (Zhang et al., 2025) งานวิจัยที่ผ่านมานำ eBPF ไปใช้ในหลายด้าน ได้แก่ การวัดจังหวะเวลาของสถานะ TCP พร้อมตรวจจับความผิดปกติด้วย CPU Overhead ต่ำกว่า 1% (Peng et al., 2026) การนับสถิติรายไอพีด้วยความละเอียดระดับต่ำกว่ามิลลิวินาที (Mei et al.,

2025) และการจำกัดอัตราแพ็กเก็ต SYN ด้วย XDP เพื่อบรรเทาการโจมตีแบบ SYN Flooding (Loureiro et al., 2024)

จากการทบทวนวรรณกรรมพบว่า งานส่วนใหญ่แยกส่วนกันระหว่างมิติในการวัดจังหวะเวลาการเชื่อมต่อ การนับสถิติ และการบรรเทาภัยคุกคาม โครงงานนี้จึงเสนอแนวคิดการนำข้อมูลเหตุการณ์ TCP ระดับเคอร์เนลมาใช้ในกระบวนการเดียวกัน ตั้งแต่การเก็บข้อมูลจากเคอร์เนลด้วย eBPF การส่งต่อผ่าน Ring Buffer ไปยังโปรแกรมภาษา Go ในระดับ User Space การสกัดคุณลักษณะ (Feature Extraction) ด้วยหน้าต่างเลื่อน (Sliding Window) และการตรวจจับความผิดปกติด้วยเกณฑ์ตัดสินใจ (Threshold-based Detection) ที่มีน้ำหนักเบา โดยบันทึกบันทึกเหตุการณ์ (Log) เฉพาะเมื่อตรวจพบความผิดปกติ เพื่อลดภาระการจัดเก็บข้อมูล ระบบนี้ออกแบบสำหรับเครื่องเดี่ยว (Single-host) และเครือข่ายท้องถิ่นขนาดเล็ก (LAN) สำหรับการทดสอบเบื้องต้น

ปัจจุบันคณะผู้จัดทำได้พัฒนาต้นแบบ (Toy Model) ที่สามารถดักจับเหตุการณ์การเปลี่ยนสถานะของ TCP Socket และเหตุการณ์ Retransmission ในระดับเคอร์เนล ส่งผ่าน Ring Buffer ไปยังโปรแกรม Go และแจ้งเตือนเมื่อพบ Handshake Latency สูงหรือมี Retransmission ผ่านหน้าจอ Terminal เพื่อใช้เป็นฐานในการพัฒนาส่วนที่เหลือของโครงงาน

คำถามวิจัยของโครงงาน คือ การใช้ eBPF เก็บข้อมูลการเชื่อมต่อ TCP ในระดับเคอร์เนลสามารถให้ข้อมูลเพียงพอต่อการตรวจจับความผิดปกติแบบเรียลไทม์ได้มากน้อยเพียงใด และมีภาระต่อระบบ (System Overhead) อยู่ในระดับที่ยอมรับได้หรือไม่

3. วัตถุประสงค์ของโครงงาน

3.1 เพื่อออกแบบและพัฒนาระบบตรวจสอบเหตุการณ์การเชื่อมต่อ TCP ในระดับเคอร์เนลด้วย eBPF และส่งข้อมูลไปยัง User Space เพื่อประมวลผลแบบเรียลไทม์

3.2 เพื่อพัฒนากลไกตรวจจับความผิดปกติของการเชื่อมต่อ TCP โดยใช้เกณฑ์ตัดสินใจร่วมกับสถิติพื้นฐานที่สกัดจากหน้าต่างเลื่อน (Sliding Window)

3.3 เพื่อสร้างระบบต้นแบบด้วยภาษา Go สำหรับรับ ประมวลผล และแสดงผลสถิติผ่านแผงควบคุม (Dashboard) พร้อมระบบจัดเก็บบันทึกเหตุการณ์ผิดปกติย้อนหลัง

3.4 เพื่อประเมินประสิทธิภาพของระบบทั้งด้านความสามารถในการตรวจจับและภาระต่อทรัพยากร (CPU/Memory) และวิเคราะห์ข้อแลกเปลี่ยน (Trade-off) ระหว่างความละเอียดของข้อมูล ความแม่นยำ และ Overhead

4. ทฤษฎีและผลงานวิจัยที่เกี่ยวข้อง

4.1 การเชื่อมต่อ TCP และสถานะของ Socket

การเชื่อมต่อ TCP ผ่านวงจรสถานะ เช่น SYN_SENT, SYN_RECV, ESTABLISHED, FIN_WAIT, CLOSE_WAIT และ CLOSE เวลาระหว่างการเปลี่ยนสถานะ เช่น จาก SYN_SENT ไป ESTABLISHED สะท้อน Handshake Latency ส่วนการส่งข้อมูลซ้ำเป็นตัวบ่งชี้ว่าแพ็กเก็ตอาจสูญหายหรือเครือข่ายมีความหน่วงสูง

4.2 เทคโนโลยี eBPF

eBPF ช่วยให้โปรแกรมขนาดเล็กทำงานที่จุดต่าง ๆ ในเคอร์เนล เช่น Tracepoint และ Kprobe โดยผ่านการตรวจสอบของ Verifier ก่อนโหลด ข้อมูลที่เก็บสามารถอยู่ใน BPF Map (เช่น Hash Map และ LRU Hash Map) และส่งออกไปยัง User Space ผ่าน Ring Buffer ซึ่งต้องคำนึงถึงข้อจำกัดของ Verifier ด้านความปลอดภัยของหน่วยความจำ (Zhang et al., 2025)

4.3 งานวิจัยที่เกี่ยวข้อง

คณะผู้จัดทำทบทวนงานวิจัยที่เกี่ยวข้องกับ eBPF/XDP จำนวน 10 ฉบับ โดยมี 3 ฉบับที่เกี่ยวข้องโดยตรงกับสถาปัตยกรรมของโครงงาน สรุปได้ดังตารางที่ 1

ตารางที่ 1 สรุปงานวิจัยที่เกี่ยวข้องและสิ่งที่นำไปประยุกต์

งานวิจัย	จุดเน้น / บทบาทของ eBPF	สิ่งที่โครงงานนำไปประยุกต์
Peng et al. (2026)	ดักจับสถานะ TCP วัด Latency แบบแยกช่วง ตรวจจับแบบผสม (Threshold + Autoencoder + CUSUM)	แนวคิดวัดจังหวะเวลาจากการเปลี่ยนสถานะ TCP และใช้ Threshold P99 เป็นกลไกตรวจจับเบื้องต้น
Mei et al. (2025)	eBPF Map แบบ LRU Hash นับสถิติรายไอพีที่ความละเอียดต่ำกว่ามิลลิวินาที	แนวคิดการเก็บสถิติต่อการเชื่อมต่อใน BPF Map และควบคุมหน่วยความจำ
Loureiro et al. (2024)	XDP ทำ Rate Limiting ด้วย Fixed Window เพื่อบรรเทา SYN Flooding	แนวคิดติดตามสถานะ SYN/ACK เพื่อคำนวณสัดส่วน Handshake ที่ไม่สมบูรณ์
Scholz et al. (2018)	เปรียบเทียบประสิทธิภาพ XDP กับ iptables/nftables	ใช้เป็นข้อมูลอ้างอิงเรื่องข้อแลกเปลี่ยนด้านประสิทธิภาพ
Liu et al. (2020)	เก็บ Metric เครือข่ายแบบ Non-intrusive ด้วย Socket Filter และ Kprobe	สนับสนุนการเก็บข้อมูลระดับเคอร์เนลโดยไม่แก้ไขแอปพลิเคชัน

งานวิจัยอื่นที่ทบทวน ได้แก่ Soldani et al. (2023) ด้านแพลตฟอร์ม Observability สำหรับ 5G/6G, Ragavan & Nadig (2025) ด้านการจัดตารางงานบน Kubernetes ที่คำนึงถึงสถานะเครือข่าย, Holik et al. (2025) ด้านสถาปัตยกรรมเครือข่าย eBPF สำหรับระบบ OT และ Puttlitz et al. (2024) ด้านการวัดผลเครือข่ายแบบ End-to-End โดยงานกลุ่มนี้ใช้เป็นแนวทางอ้างอิงสำหรับการต่อยอดในอนาคต

4.4 การตรวจจับความผิดปกติด้วยเกณฑ์ตัดสินใจ

Peng et al. (2026) ใช้ Static Threshold ที่คำนวณจากเปอร์เซ็นไทล์ที่ 99 ของช่วงเวลาในสภาวะปกติ คือ

$$THR_x = P_{99}(T_x)$$

เป็นชั้นตรวจจับแรก ก่อนเสริมด้วย Autoencoder และ CUSUM โครงงานนี้เลือกใช้เฉพาะชั้น Threshold เนื่องจากอธิบายและวัดผลได้ง่าย และเหมาะกับขอบเขตงานระดับปริญญาตรี โดยออกแบบให้ต่อยอดไปสู่วิธีที่ซับซ้อนขึ้นได้ในอนาคต

5. วิธีดำเนินการวิจัย

5.1 ศึกษาค้นคว้าทฤษฎีและเครื่องมือ

ศึกษา TCP State Machine, eBPF (Tracepoint, BPF Map, Ring Buffer), ภาษา Go และไลบรารี cilium/ebpf รวมถึงทบทวนงานวิจัยที่เกี่ยวข้อง

5.2 กำหนดขอบเขตและสภาพแวดล้อม

พัฒนาและทดสอบบนระบบปฏิบัติการ Linux เครื่องเดี่ยว (Single-host) และเครือข่ายท้องถิ่นขนาดเล็ก (LAN) โดยใช้โปรแกรม eBPF ที่เขียนด้วยภาษา C คอมไพล์ด้วย clang/LLVM และโปรแกรมฝั่ง User Space ที่เขียนด้วยภาษา Go ร่วมกับไลบรารี cilium/ebpf ซึ่งใช้เครื่องมือ bpf2go ในการสร้างโค้ดเชื่อมต่อระหว่างสองฝั่งโดยอัตโนมัติ สภาพแวดล้อมที่ใช้ในการพัฒนาและทดลองสรุปได้ตามตารางที่ 2

ตารางที่ 2 สภาพแวดล้อมการพัฒนาและทดลอง

รายการ	รายละเอียด
ระบบปฏิบัติการ	Fedora Linux 44 (KDE Plasma, Wayland)
Linux Kernel	7.2.7-200.fc44.x86_64
หน่วยประมวลผล (CPU)	Intel Core i5-8350U (4 คอร์ 8 เธรด) ความเร็ว 3.60 GHz

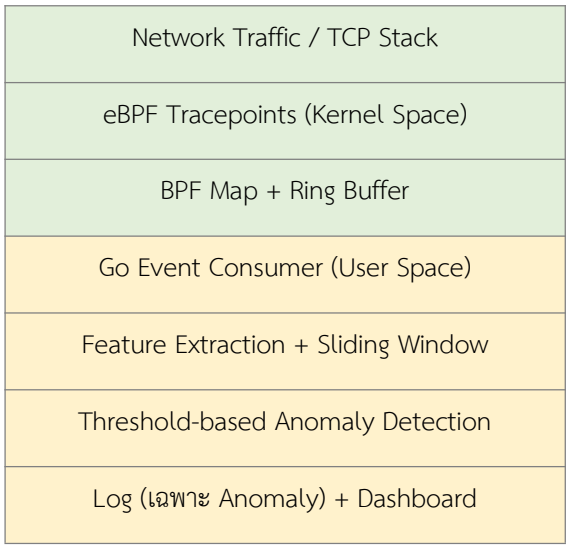
หน่วยความจำ (RAM)	7.57 GiB
อินเทอร์เฟซเครือข่าย	Wi-Fi (wlp2s0)
ภาษา Go	go1.26.8 linux/amd64
ไลบรารี cilium/ebpf	v0.22.0
คอมไพเลอร์ eBPF	clang/LLVM รุ่น 22.1.8 (Fedora 22.1.8-4.fc44)
BTF	/sys/kernel/btf/vmlinux

ในการทดลองจะบันทึกรุ่นของซอฟต์แวร์และเงื่อนไขของเครื่องทดสอบไว้ทุกครั้ง การรันโปรแกรม eBPF ต้องใช้สิทธิ์ root และผลการทดลองในรายงานนี้ใช้ได้กับสภาพแวดล้อมข้างต้นเท่านั้น ไม่ได้ทดสอบกับเคอร์เนลรุ่นอื่น

5.3 ออกแบบสถาปัตยกรรมระบบ

ระบบแบ่งหน้าที่เป็นชั้นชัดเจน โดยให้เคอร์เนลเก็บเฉพาะข้อมูลที่จำเป็น และย้ายการประมวลผลที่ซับซ้อนไปทำใน User Space ดังภาพที่ 1

ภาพที่ 1 สถาปัตยกรรมเชิงแนวคิดของระบบ (สีเขียว: Kernel Space, สีเหลือง: User Space)



หมายเหตุการปรับแบบ: เอกสารรายงานสัมมนาเดิมระบุจุดดักจับเป็น kprobe/kretprobe (เช่น tcp_v4_connect, tcp_set_state) แต่ต้นแบบปัจจุบันใช้ Tracepoint (sock/inet_sock_set_state และ tcp/tcp_retransmit_skb) ซึ่งให้ข้อมูลสถานะเก่า/ใหม่และที่อยู่ไอพี/พอร์ตมาใน Context ของ Event โดยตรง ทั้งนี้ Tracepoint และ kprobe ยังอยู่ในกลุ่มการเก็บข้อมูลระดับเคอร์เนลแบบ Non-intrusive ตามแนวคิดของ Peng et al. (2026) และ Liu et al. (2020) นอกจากนี้ BPF Map ที่ใช้ในต้นแบบเป็น Hash Map ที่มี Socket Pointer เป็น Key ส่วนการใช้ LRU Hash Map ตามแนวคิดของ Mei et al. (2025) จะพิจารณาในขั้นพัฒนาต่อไป

5.4 พัฒนาระบบ

ดำเนินการเป็นขั้นตอน จาก Kernel สู่ Data Pipeline ไปยัง Detection และแสดงออกมายัง Visualization เพื่อให้ทดสอบแต่ละส่วนได้ก่อนนำมาประกอบกัน สถานะการพัฒนาปัจจุบันสรุปได้ตามตารางที่ 3

ตารางที่ 3 สถานะการพัฒนาระบบ ณ ปัจจุบัน

ส่วนประกอบ	รายละเอียด	สถานะ
Kernel: ดักจับการเปลี่ยนสถานะ TCP	Tracepoint sock/inet_sock_set_state วัด Handshake Latency (SYNto ESTABLISHED) และอายุการเชื่อมต่อ	ทำแล้ว (ต้นแบบ)
Kernel: ดักจับ Retransmission	Tracepoint tcp/tcp_retransmit_skb	ทำแล้ว (ต้นแบบ)
Kernel: ส่งข้อมูลขึ้น User Space	BPF Ring Buffer ขนาด 256 KB; BPF Hash Map เก็บเวลาต่อ Socket (สูงสุด 10,240 รายการ)	ทำแล้ว (ต้นแบบ)
User: รับและแสดงผล Event	Go อ่าน Ring Buffer แสดงผลบน Terminal (เวลา, โปรเซส, ที่อยู่ต้นทาง/ปลายทาง, การเปลี่ยนสถานะ, ระยะเวลา)	ทำแล้ว (ต้นแบบ)
User: ตรวจจับด้วย Threshold	เกณฑ์คงที่: Handshake Latency > 50 ms และ Retransmission > 0	ทำแล้ว (เกณฑ์คงที่)
User: Feature Extraction และ Sliding Window	สรุปสถิติต่อหน้าต่างเวลา เช่น อัตรา Retransmission, สัดส่วน Handshake ไม่สมบูรณ์	ยังไม่ทำ
Threshold แบบ P99 จากข้อมูลปกติ	เก็บข้อมูล Baseline และคำนวณเกณฑ์ใหม่แทนค่าคงที่	ยังไม่ทำ

ส่วนประกอบ	รายละเอียด	สถานะ
Logging และ Dashboard	บันทึกเฉพาะเหตุการณ์ผิดปกติ และแสดงสถิติแบบเรียลไทม์	ยังไม่ทำ
การทดลองและประเมินผล	วัด Detection Rate, False Positive, CPU/Memory Overhead	ยังไม่ทำ

โครงสร้างข้อมูลที่ส่งจากเคอร์เนล (struct tcp_event) ประกอบด้วย เวลา (timestamp) ระยะเวลา (duration) ไอพีและพอร์ตต้นทาง/ปลายทาง สถานะเก่าและใหม่ ประเภท Event และชื่อโปรเซส ปัจจุบันรองรับเฉพาะ IPv4

5.5 ออกแบบและดำเนินการทดลอง

ทดสอบในสภาพแวดล้อมเครื่องเดี่ยวหรือ LAN โดยสร้างทั้ง Traffic ปกติและ Traffic ที่มีพฤติกรรมผิดปกติ ทำซ้ำหลายรอบและบันทึกผลทุกครั้ง ตัวอย่างสถานการณ์ทดสอบ

- Traffic ปกติ: การเชื่อมต่อเว็บด้วย curl (ใช้ใน test_traffic.sh แล้ว)
- Retransmission/SYN Timeout: เชื่อมต่อไปยังที่อยู่ที่ไม่ตอบสนอง เช่น 192.0.2.1 พอร์ต 81 (ใช้ใน test_traffic.sh แล้ว)
- Handshake Latency สูง: จำลองความหน่วงของเครือข่ายด้วยเครื่องมือ เช่น tc netem (วางแผน)
- การเชื่อมต่อจำนวนมากในเวลาสั้นและ Handshake ไม่สมบูรณ์: (วางแผน ต้องเพิ่ม Feature ในชั้น Sliding Window ก่อน)

ตัวชี้วัด ได้แก่ Detection Rate, False Positive Rate, ความหน่วงในการประมวลผล Event, การใช้ CPU และหน่วยความจำของระบบเมื่อเปิดและปิดการตรวจสอบ และเปรียบเทียบกับแนวทางอ้างอิงตามระเบียบวิธีของ Mei et al. (2025)

5.6 สรุปผล เขียนรายงาน และนำเสนอผลงาน

วิเคราะห์ผลการทดลอง สรุปข้อแลกเปลี่ยนระหว่างความละเอียดของข้อมูล ความแม่นยำ และภาระต่อระบบ จัดทำรายงานฉบับสมบูรณ์ และนำเสนอผลงานต่ออาจารย์ที่ปรึกษาและคณะกรรมการ

6. ขอบเขตและข้อจำกัดของการวิจัย

ขอบเขต

6.1 พัฒนาระบบต้นแบบสำหรับ Linux เครื่องเดี่ยวและเครือข่าย LAN ขนาดเล็ก ดักจับเหตุการณ์ TCP ด้วย eBPF ในเคอร์เนลและประมวลผลด้วยภาษา Go

6.2 ตรวจจับความผิดปกติด้วย Threshold-based Detection เป็นหลัก ครอบคลุมความผิดปกติเชิงจังหวะเวลา (Handshake Latency) เชิงการส่งซ้ำ (Retransmission) และเชิงปริมาณ (Handshake ไม่สมบูรณ์) เมื่อพัฒนาครบ

6.3 แสดงผลผ่าน Dashboard และบันทึก Log เฉพาะเหตุการณ์ผิดปกติ

ข้อจำกัด

6.4 ไม่รวมแบบจำลอง Machine Learning แบบเต็มรูปแบบ (เช่น Autoencoder) และไม่ครอบคลุมการบล็อกหรือทิ้งแพ็กเก็ตโดยตรง (ระบบตรวจจับและแสดงผลเท่านั้น)

6.5 ต้นแบบปัจจุบันรองรับเฉพาะ IPv4 และ TCP ต้องรันด้วยสิทธิ์ root และต้องใช้เคอร์เนลที่รองรับ eBPF/BTF และ Tracepoint ที่เกี่ยวข้อง

6.6 ไม่ทดสอบบนเครือข่ายความเร็วสูงระดับ 100 Gbps หรือสภาพแวดล้อม Kubernetes/Data Center ผลการประเมินใช้ได้กับสภาพแวดล้อมทดลองที่ระบุเท่านั้น

6.7 Ring Buffer มีขนาดจำกัด หากปริมาณ Event สูงมากอาจมี Event ถูกทิ้ง

7. สถานที่ทำวิจัย

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์ มหาวิทยาลัยขอนแก่น

8. ประโยชน์ที่คาดว่าจะได้รับ

8.1 ได้ระบบต้นแบบที่ตรวจสอบเหตุการณ์ TCP ระดับเคอร์เนลแบบเรียลไทม์โดยมีภาระต่อระบบต่ำ

8.2 ผู้ดูแลระบบติดตามพฤติกรรม TCP และสถานะการจราจรผ่าน Dashboard ได้สะดวก

8.3 ลดปริมาณข้อมูลที่ต้องจัดเก็บ โดยบันทึกเฉพาะเหตุการณ์ผิดปกติ

8.4 ได้องค์ความรู้เชิงทดลองเรื่อง Trade-off ระหว่างความละเอียดของข้อมูล ความแม่นยำ และ Overhead

8.5 ได้แนวทางและสถาปัตยกรรมตัวอย่างสำหรับต่อยอดระบบเฝ้าระวังเครือข่ายที่เหมาะกับองค์กรขนาดเล็ก

9. แผนและระยะเวลาดำเนินการ

ตารางที่ 3 แผนและระยะเวลาดำเนินการ (ร่าง: ปรับตามปฏิทินการศึกษาและช่วงที่เสนอเค้าโครง)

การดำเนินงาน	2569				2570				
	ก.ย.	ต.ค.	พ.ย.	ธ.ค.	ม.ค.	ก.พ.	มี.ค.	เม.ย.	พ.ค.
1. ศึกษาทฤษฎี ทบทวนวรรณกรรม และออกแบบสถาปัตยกรรม									
2. พัฒนาและทดสอบส่วน Kernel Space (Tracepoint, Ring Buffer)									
3. พัฒนา User Space Pipeline (Feature Extraction, Sliding Window)									
4. พัฒนา Threshold Detection (P99) และ Logging									
5. พัฒนา Dashboard									
6. ทดลองและประเมินผล (Detection, Overhead)									
7. สรุปผล เขียนรายงาน และนำเสนอผลงาน									

10. งบประมาณ

หมวดวัสดุอุปกรณ์

- ค่าวัสดุสำนักงาน (กระดาษ ปากกา และอื่น ๆ) 200 บาท
- ค่าวัสดุคอมพิวเตอร์ (ใช้เครื่องแลปท็อปส่วนตัว, สายLAN) 100 บาท

หมวดค่าใช้สอย

- ค่าถ่ายเอกสาร 200 บาท
- ค่าจัดรูปเล่ม 400 บาท

รวมทั้งสิ้น 900 บาท

11. เอกสารอ้างอิง

Holik, F., Jouet, S., & Pezaros, D. (2025). An eBPF-based programmable network architecture for OT digital resilience use-cases. In 2025 IEEE Symposium on Computers and Communications (ISCC). IEEE. https://doi.org/10.1109/ISCC65549.2025.11325939

Zhang, X., Wang, Y., Chen, L., & Li, J. (2025). SoK: Challenges and paths toward memory safety for eBPF. In 2025 IEEE Symposium on Security and Privacy (SP) (pp. 1234–1248). IEEE. https://doi.org/10.1109/SP61157.2025.00134

Liu, C., Cai, Z., Wang, B., Tang, Z., & Liu, J. (2020). A protocol-independent container network observability analysis system based on eBPF. In Proceedings of the 2020 IEEE 26th International Conference on Parallel and Distributed Systems (ICPADS) (pp. 697–702). IEEE. https://doi.org/10.1109/ICPADS51040.2020.00099

Loureiro, L., Pereira, V., Cruz, T., & Simões, P. (2024). Enhancing 5G core security with eBPF/XDP. In NOMS 2024 IEEE Network Operations and Management Symposium. IEEE. https://doi.org/10.1109/NOMS59830.2024.10575063

Mei, X., Chen, J., Singh, A., Baldin, I., & Lawrence, D. (2025). eCounter: Inline per-IP network monitoring at millisecond resolution via eBPF. In SC Workshops 2025 (pp. 908–918). ACM. https://doi.org/10.1145/3731599.3767445

Peng, T., Lin, C.-H., Wei, Y.-C., Chen, B.-C., & Yeh, J.-Y. (2026). Low-overhead passive resilience monitoring for AIoT gateways: End-to-end anomaly detection via eBPF timing traces and autoencoders. IEEE Access, 14, 61753–61771. https://doi.org/10.1109/ACCESS.2026.3685156

Puttlitz, M., Parizotto, R., & Schaeffer-Filho, A. (2024). P4NetIntel: End-to-end network telemetry with eBPF and XDP. In 2024 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN). IEEE. https://doi.org/10.1109/NFV-SDN61811.2024.10807468

Ragavan, V. K. K., & Nadig, D. (2025). NAS: A novel network-aware Kubernetes scheduling framework using eBPF service mesh. In ICC 2025 - IEEE International Conference on Communications. IEEE. https://doi.org/10.1109/icc52391.2025.11161235

Scholz, D., Raumer, D., Emmerich, P., Kurtz, A., Lesiak, K., & Carle, G. (2018). Performance implications of packet filtering with Linux eBPF. In 30th International Teletraffic Congress (ITC 30) (pp. 209–217). IEEE. https://doi.org/10.1109/ITC30.2018.00039

Soldani, D., Nahi, P., Bour, H., Jafarizadeh, S., Soliman, M. F., Di Giovanna, L., Monaco, F., Ognibene, G., & Risso, F. (2023). eBPF: A new approach to cloud-native observability, networking and security for current (5G) and future mobile networks (6G and beyond). IEEE Access, 11, 57174–57202. https://doi.org/10.1109/ACCESS.2023.3281480

ลงชื่อผู้ทำโครงงาน .....

(นางสาวปภาวรินทร์ นาเมืองรักษ์)

วันที่ .....

ลงชื่อผู้ทำโครงงาน .....

(นายเมธัส มณีวิจิตร)

วันที่ .....

การตรวจสอบจากอาจารย์ที่ปรึกษาโครงงาน

( ) ตรวจสอบแล้ว เห็นชอบตามเสนอ

( ) อื่น ๆ .....

(ลงชื่อ) .....

(.....)

วันที่ .....